

② 事务在运行过程中被强行停止。

在第一种情况下,数据库管理系统必须保证多个事务的交叉运行不影响这些事务的隔离性;在第二种情况下,数据库管理系统必须保证被强行终止的事务对数据库和其他事务没有任何影响。

这些就是数据库管理系统中并发控制机制和恢复机制的责任。

11.2 数据库恢复概述

尽管数据库系统中采取了各种保护措施来防止数据库的安全性和完整性被破坏,保证并发事务的正确执行,但是计算机系统中硬件的故障、软件的错误、操作员的失误以及恶意的破坏仍是不可避免的。这些故障轻则造成运行事务非正常中断,影响数据库中数据的正确性;重则破坏数据库,使数据库中全部或部分数据丢失。因此数据库管理系统必须具有把数据库从错误状态恢复到某一已知的正确状态(亦称为一致性状态或完整性状态)的功能,这就是数据库的恢复。恢复子系统是数据库管理系统的一个重要组成部分,而且还相当庞大,常常占整个系统代码的10%以上。数据库系统所采用的恢复技术是否行之有效,不仅对系统的可靠程度起着决定性作用,而且对系统的运行效率也有很大影响,是衡量系统性能优劣的重要指标。

11.3 故障的种类

数据库系统中可能发生各种各样的故障,大致可以分以下几类。

1. 事务内部的故障

事务内部的故障有些是可以通过事务程序本身发现的,有些则是非预期的,不能由事务程序处理。

例如,下述语句为银行转账事务,这个事务把一笔金额从一个账户甲转给另一个账户乙。

```
BEGIN TRANSACTION
    读账户甲的余额 BALANCE1;
    BALANCE1=BALANCE1-AMOUNT;    /* AMOUNT 为转账金额 */
    IF (BALANCE1 < 0) THEN
        |打印'金额不足,不能转账';    /* 事务内部可能造成事务被回滚的情况 */
        ROLLBACK;|    /* 撤销刚才的修改,恢复事务 */
    ELSE
        |读账户乙的余额 BALANCE2;
        BALANCE2=BALANCE2+AMOUNT;
        写回 BALANCE2;
        COMMIT;|
```

这个例子所包含的两个更新操作要么全部完成,要么全部不做,否则就会使数据库处于不一致状态,例如可能出现只把账户甲的余额减少而没有把账户乙的余额增加的情况。

在这段程序中若产生账户甲余额不足的情况,应用程序可以发现并让事务回滚,撤销已做的修改,将数据库恢复到正确状态。

事务内部更多的故障是非预期的,不能由应用程序处理,如运算溢出、并发事务发生死锁而被选中撤销该事务、违反某些完整性限制而被终止等。本书后续内容中,事务故障仅指这类非预期的故障。

事务故障意味着事务没有达到预期的终点(COMMIT 或显式的 ROLLBACK),因此数据库可能处于不正确状态。恢复程序要在不影响其他事务运行的情况下强行回滚该事务,即撤销该事务已做出的任何对数据库的修改,使得该事务好像根本没有启动一样。这类恢复操作称为**事务撤销(UNDO)**。

2. 系统故障

系统故障是指造成系统停止运转的任何事件,使得系统要重新启动。例如,特定类型的硬件错误(CPU 故障)、操作系统故障、数据库管理系统代码错误、系统断电、导致系统崩溃的计算机病毒等。这类故障影响正在运行的所有事务,但不破坏数据库。此时主存内容,尤其是数据库缓冲区(在内存)中的内容都被丢失,所有运行事务都非正常终止。发生系统故障时,一些尚未完成的事务的结果可能已送入物理数据库,从而造成数据库可能处于不正确的状态。为保证数据一致性,需要清除这些事务对数据库的所有修改。

恢复子系统必须在系统重新启动时让所有非正常终止的事务回滚,强行撤销所有未完成的事务。

另一方面,发生系统故障时,有些已完成的事务的结果可能有一部分甚至全部留在缓冲区,尚未写回到磁盘上的物理数据库中,系统故障使得这些事务对数据库的修改部分或全部丢失,这也会使数据库处于不一致状态,因此应将这些事务已提交的结果重新写入数据库。所以系统重新启动后,恢复子系统除需撤销所有未完成的事务外,还需要**重做(REDO)**所有已提交的事务,以将数据库真正恢复到一致状态。

3. 介质故障

通常把系统故障称为**软故障(soft fault)**、介质故障称为**硬故障(hard fault)**。硬故障指外存故障,如磁盘损坏、磁头碰撞、瞬时强磁场干扰、破坏硬盘数据的计算机病毒等。这类故障将破坏数据库或部分数据库,并影响正在存取这部分数据的所有事务。这类故障比前两类故障发生的可能性小得多,但破坏性最大,需要借助存储在其他地方的数据备份来恢复数据库。

除此之外,如果计算机染上了“勒索病毒”,虽然这种病毒不破坏外存设备,但会对文件进行加密,使数据不可访问,因而对数据库造成的影响与介质故障类似,也只能借助数据备份来恢复数据库。

总结各类故障对数据库的影响,有两种可能性:一是数据库本身被破坏;二是数据库没有

被破坏,但数据可能不正确,这是由于事务的运行被非正常终止造成的。

恢复的基本原理十分简单。可以用一个词来概括——冗余。也就是说,数据库中任何一部分被破坏或不正确的数据可以根据存储在系统别处的冗余数据来重建。尽管恢复的基本原理很简单,但实现技术的细节却相当复杂。下面略去一些细节,介绍数据库恢复的实现技术。

11.4 恢复的实现技术

恢复机制涉及两个关键问题,即如何建立冗余数据,以及如何利用这些冗余数据实施数据库恢复。

建立冗余数据最常用的技术是数据转储(dump)和登记日志文件。通常在一个数据库系统中这两种方法是一起使用的。

11.4.1 数据转储

数据转储是数据库恢复中采用的基本技术。所谓数据转储,即数据库管理员定期将整个数据库复制到磁带、磁盘或其他存储介质上保存的过程。这些备用的数据称为后备副本或后援副本。

当数据库遭到破坏后可以将后备副本重新装入,但重装后备副本只能将数据库恢复到转储时的状态,要想恢复到故障发生时的状态,必须重新运行自转储以后的所有更新事务。例如,在图 11.1 中,系统在 T_a 时刻停止运行事务,进行数据库转储,在 T_b 时刻转储完毕,得到 T_b 时刻的数据库一致性副本。系统运行到 T_f 时刻发生故障。为恢复数据库,首先由数据库管理员重装数据库后备副本,重装的后备副本是 T_a 时刻转储的副本,将数据库恢复至 T_b 时刻的状态,然后重新运行自 $T_b \sim T_f$ 时刻所有已提交的更新事务,这样就把数据库恢复到故障发生前的一致状态了。

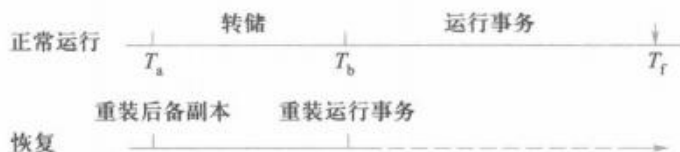


图 11.1 转储和恢复示例

数据转储是十分耗费时间和资源的,不能频繁进行。数据库管理员应根据数据库使用情况确定一个适当的转储周期。

数据转储按数据库的状态可分为静态转储和动态转储。

静态转储是在系统中无运行事务时进行的转储操作。静态转储操作开始的时刻数据库处于一致性状态,在转储期间不允许(或不存在)对数据库的任何存取、修改活动。显然,静态转储得到的一定是一个数据一致性的副本。